Day 148: Stationarity and Differencing: the Augmented Dickey-Fuller Test, from Scratch

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 22, Day 2 | Complete study guide

Watch: What is Stationarity (Aric LaBarr) | Time Series Talk: Augmented Dickey Fuller Test + Code (ritvikmath)

Script: `day148_stationarity_differencing.py` · every number in this guide is copied by program from `day148_run_output.txt`, the real output of that script.

A stationary series behaves the same in every stretch of time, and most classical forecasting models assume one. Day 147's series are not stationary. Today differences them (each month minus an earlier month) and tests the result with two tests written from scratch: the augmented Dickey-Fuller (ADF) test, matching statsmodels to 1.0e-11, and KPSS (1.1e-16). The ADF test needs its own table of critical values; a simulation of 20,000 random walks reproduces MacKinnon's published one (-2.874 against -2.867 at 5%) and shows that the usual normal table would wrongly reject 0.452 of the time. On the real series the two tests often disagree, and near a unit root the ADF test is weak: it caught an AR(1) with phi 0.95 only 0.120 of the time in 100 steps.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

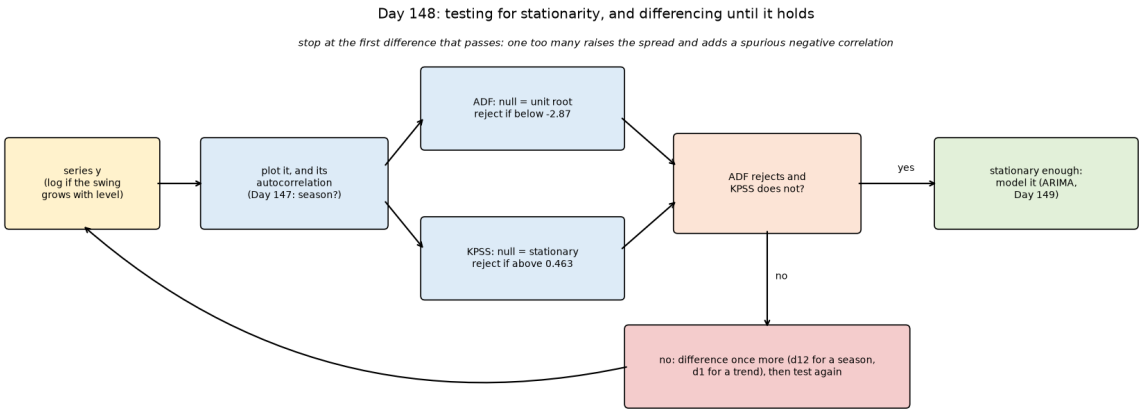
A series is **stationary** when its mean, its spread and the way each value relates to earlier ones do not change over time. A **unit root** is the opposite extreme: a series that wanders like a random walk, where each step's change is permanent. **Differencing** replaces the series by its changes; it turns a random walk into noise and a straight trend into a constant.

Day 147 split a series into trend and season. Today removes them by differencing instead, which is what ARIMA (Day 149) does.

REAL-WORLD SCENARIO

A person's daily weight is roughly stationary around their usual weight: a heavy day is followed by a return to normal. Their savings balance is not: a big purchase lowers it for good. A forecast of tomorrow's weight should lean towards the usual value; a forecast of tomorrow's balance should start from today's. Telling which kind of series you have is today's job.

SYSTEM DIAGRAM



Test, difference, test again; stop at the first difference that passes.

Today's part for the week's build

`day148_exercise.py` is today's parts (the autocorrelation function, differencing, the Dickey-Fuller statistic for many series at once, a simulated table of critical values and its p-value, the ADF test with lags chosen by AIC, and KPSS), checked by hand: the autocorrelation of 1 to 5, two differences, the statistic against a plain least-squares fit, and KPSS on noise and on a random walk.

ACTUAL OUTPUT

```
acf of 1, 2, 3, 4, 5: centred -2 -1 0 1 2, sum of squares 10; lag 1 = 4 / 10 = 0.4, lag 2 = -1 /
    10 = -0.1; acf() gives 0.4, -0.1
difference of 1, 4, 9, 16: [3, 5, 7] (lag 1), [8, 12] (lag 2)
Dickey-Fuller statistic of one random walk: plain least squares -1.390251, df_stats -1.390251, adf
    with no extra lags -1.390251
simulated table, length 100, 5,000 walks: 5% point -2.919 (MacKinnon's published value for 100:
    about -2.89)
mc_pvalue(2.5, table 1 2 3 4) = 0.50 (two of four values are at or below 2.5)
KPSS: white noise 0.104, a random walk 1.776 (5% critical value 0.463: the walk is rejected as
    stationary)
```

Data: Day 147's two series, loaded by its parts: monthly CO2 at Mauna Loa (526 months) and euro-area orders for electrical equipment (257 months), the orders on a log scale because their seasonal swing grows with their level (Day 147, section 4). Four simulated series of 500 steps, whose behaviour is known, serve as controls.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. What stationary means

DEFINITION — start here, no prior knowledge assumed

Split a series in half and compare the halves. For a stationary series their means and spreads should be similar; for a wandering or trending one they are not. The **autocorrelation** at lag k is the correlation between the series and itself k steps later: near 0 for noise, near 1 for a series that changes slowly.

REAL-WORLD SCENARIO

A thermostat-controlled room has a stationary temperature; a room whose heater was left on has a trending one. Comparing the first and second halves of the day tells them apart.

SPECIFICATION

White noise	independent normal values, mean 0, spread 1
AR(1), phi 0.9	$y_t = 0.9\ y_{t-1}$ + noise: pulled back towards 0 each step, so stationary
Random walk	$y_t = y_{t-1}$ + noise: a unit root
Trend + noise	0.02 t + noise: stationary around a straight line, not around a constant

ACTUAL OUTPUT

```
a stationary series has the same mean and spread in every stretch; compare the first and second halves
                series  mean, 1st half  2nd half  std, 1st half  2nd half  acf lag 1  lag 12
          white noise           0.032    -0.082          0.996     0.997     -0.011   0.022
       AR(1), phi 0.9          -1.539     0.231          1.829     2.338      0.904   0.440
          random walk           9.031    15.909          8.113     3.877      0.988   0.868
        trend + noise           2.515     7.461          1.744     1.743      0.887   0.826
                  CO2         324.770   354.468          6.728     9.928      0.993   0.935
           log orders           4.547     4.650          0.176     0.125      0.659   0.744
white noise keeps its mean and spread; the AR(1) is stationary too, but with phi 0.9 it has long memory, so 250 steps
are a short sample and its halves still differ; the random walk, the trend and CO2 drift away
Actual output from this lesson's run.
```

What it means

White noise kept its mean and spread (0.996 and 0.997). The random walk's halves had means 9.031 and 15.909, and CO2's 324.770 and 354.468. The AR(1) shows the catch: it is stationary by construction, yet its halves differ (-1.539 and 0.231), because with phi 0.9 it has long memory and 250 steps are a short sample. Looking is not enough; that is why tests exist.

WHY CHOOSE THIS?

Comparing halves and plotting is quick and catches obvious trends.

WHY NOT THIS? (TRADEOFFS)

It cannot tell a slowly mean-reverting series from a random walk in a short sample, as the AR(1) shows.

2. The autocorrelation function

DEFINITION — start here, no prior knowledge assumed

The autocorrelation function (ACF) lists the autocorrelation at lags 1, 2, 3, ... For an AR(1) it should fall like phi^k; for a random walk it falls very slowly; for a seasonal monthly series it has a peak at lag 12.

REAL-WORLD SCENARIO

How similar is today's weather to yesterday's? To last week's? To the same day last year? The ACF answers that for every gap at once.

MATHEMATICS

$$r_k = \Sigma_t (y_t - m)(y_{t+k} - m) \,/\, \Sigma_t (y_t - m)^2$$

m = the mean of the series; $r_0 = 1$

IMPLEMENTATION

```
def acf(x, nlags):
    x = np.asarray(x, float) - np.mean(x); d = np.sum(x * x)
    return np.array([1.0] + [np.sum(x[:-k] * x[k:]) / d for k in range(1, nlags + 1)])
```

ACTUAL OUTPUT

```
largest difference from statsmodels' acf, lags 0-40, five series: 6.7e-16
AR(1) with phi 0.9: lags 1-5 0.904 0.826 0.763 0.699 0.648; theory phi^k: 0.900 0.810 0.729 0.656 0.590
random walk: lag 1 0.988, lag 25 0.713, lag 100 0.094 -- it fades slowly, the mark of a unit root
Actual output from this lesson's run.
```

What it means

The from-scratch ACF matches statsmodels' (6.7e-16). The AR(1)'s autocorrelations (0.904 0.826 0.763 0.699 0.648) follow the theory (0.900 0.810 0.729 0.656 0.590) loosely, as a sample of 500 allows. The random walk's falls slowly: 0.988 at lag 1, still 0.713 at lag 25.

WHY CHOOSE THIS?

One picture shows memory, trend and season at once, and Day 149 uses it to choose a model's order.

WHY NOT THIS? (TRADEOFFS)

Sample autocorrelations are noisy, and a trend makes every lag look large, hiding everything else.

3. Why the Dickey-Fuller test needs its own table

DEFINITION — start here, no prior knowledge assumed

The Dickey-Fuller idea: regress each change on the level before it. If the series is pulled back towards a mean, a high level is followed by a fall, so the coefficient (gamma) is negative; for a random walk it is 0. The test statistic is gamma divided by its standard error, the usual t statistic.

The catch: when the series really is a random walk, this statistic does not follow the usual t or normal distribution. Its distribution has to be found another way; here, by simulating random walks and computing the statistic for each.

REAL-WORLD SCENARIO

A ruler that is always 2 mm short gives consistent but wrong readings; the normal table is such a ruler for this statistic. Simulating the case being tested measures how far off it is.

MATHEMATICS

$$\Delta y_t = a + \gamma y_{t-1} + e_t \quad \textit{(with a trend: } + b\,t\textit{)}$$

$$\textit{statistic} = \gamma / se(\gamma); \quad \textit{unit root: } \gamma = 0; \quad \textit{stationary: } \gamma < 0$$

IMPLEMENTATION

```
def df_stats(Y, regression="c"):
    n = len(Y) - 1
    D = np.diff(Y, axis=0); L = Y[:-1]
    det = np.ones((n, 1)) if regression == "c" else np.column_stack([np.ones(n), np.arange(1, n +
        1)])
    Q, _ = np.linalg.qr(det)
    D = D - Q @ (Q.T @ D); L = L - Q @ (Q.T @ L)
    g = np.sum(L * D, axis=0) / np.sum(L * L, axis=0)
    s2 = np.sum((D - g * L) ** 2, axis=0) / (n - det.shape[1] - 1)
    return g / np.sqrt(s2 / np.sum(L * L, axis=0))
def df_table(n, regression="c", sims=SIMS, seed=0):
    Y = np.cumsum(np.random.RandomState(seed).standard_normal((n, sims)), axis=0)
    return np.sort(df_stats(Y, regression))
```

ACTUAL OUTPUT

```
20,000 random walks of 500 steps; for each, regress the change on the previous level and take the t statistic
                  regression       1%       5%      10%   share below -1.645 (a 5% one-sided normal test)
         constant, simulated   -3.439   -2.874   -2.576   0.452
         constant, MacKinnon   -3.444   -2.867   -2.570
 constant + trend, simulated   -3.933   -3.405   -3.135   0.774
 constant + trend, MacKinnon   -3.977   -3.419   -3.132
         normal distribution   -2.326   -1.645   -1.282
(simulation: 0.9 s)
Actual output from this lesson's run.
```

What it means

The simulated 5% point, -2.874, agrees with MacKinnon's published -2.867; with a trend term, -3.405 against -3.419. The normal table's -1.645 would call 0.452 of genuine random walks stationary (with a trend, 0.774), instead of 0.05. The whole simulation took 0.9 s, because all 20,000 regressions are done as a few matrix operations.

WHY CHOOSE THIS?

A simulation shows where the table comes from and can be run for any sample size or regression.

WHY NOT THIS? (TRADEOFFS)

It is only as precise as its number of runs: 20,000 cannot resolve p-values below 1 in 20,000.

4. The ADF test from scratch, on the real series

DEFINITION — start here, no prior knowledge assumed

The **augmented** test adds lagged changes to the regression, so that short-term memory in the changes is not mistaken for mean reversion. How many lags? Each length from 0 to a maximum is fitted on the same rows, and the one with the smallest **AIC** (a fit score that charges for each extra coefficient) is kept, as statsmodels does.

REAL-WORLD SCENARIO

A doctor taking a pulse waits for the patient to settle first; the extra lags are the settling time, so that a short burst is not mistaken for the patient's resting state.

MATHEMATICS

$$\Delta y_t = a + \gamma y_{t-1} + \delta_1 \Delta y_{t-1} + \ldots + \delta_p \Delta y_{t-p} + e_t$$

$AIC = m \log(SSR / m) + 2k$ *(m rows, k coefficients, SSR = sum of squared residuals); p from 0 to 12* $(n / 100)^{1/4}$

IMPLEMENTATION

```
def adf(y, regression="c", maxlag=None):
    y = np.asarray(y, float); n = len(y); k0 = 1 if regression == "c" else 2
    if maxlag is None:
        maxlag = min(n // 2 - k0 - 1, int(np.ceil(12 * (n / 100) ** 0.25)))
    dy = np.diff(y)
    def fit(p, start):
        r = np.arange(start, len(dy))
        det = [np.ones(len(r))] + ([np.arange(1, len(r) + 1)] if regression == "ct" else [])
        X = np.column_stack([y[r]] + [dy[r - i] for i in range(1, p + 1)] + det)
        b, ssr = np.linalg.lstsq(X, dy[r], rcond=None)[:2]; ssr = float(ssr[0]) if len(ssr) else
            float(np.sum((dy[r] - X @ b) ** 2))
        return b, ssr, X, len(r)
    aic = []
    for p in range(maxlag + 1):
        _, ssr, X, m = fit(p, maxlag); aic.append((m * np.log(ssr / m) + 2 * X.shape[1], p))
    p = min(aic)[1]
    b, ssr, X, m = fit(p, p)
    se = np.sqrt(ssr / (m - X.shape[1]) * np.linalg.inv(X.T @ X)[0, 0])
    return b[0] / se, p, m
```

ACTUAL OUTPUT

```
ADF: regress the change on the previous level, lagged changes (number chosen by AIC) and a constant; a very negative
statistic is evidence against a unit root. p: from this lesson's simulated table, and from MacKinnon's (statsmodels)
              series      ADF  statsmodels  lags  p, simulated  p, MacKinnon  acf 1  acf 12      std
                 CO2    2.233        2.233    14        1.0000        0.9989  0.993   0.935  17.1002
             CO2, d1   -4.752       -4.752    13        0.0001        0.0001  0.704   0.922   1.1849
            CO2, d12   -4.289       -4.289    13        0.0004        0.0005  0.810   0.044   0.6735
        CO2, d1 d12   -8.398       -8.398    15      < 0.0001        0.0000 -0.297  -0.409   0.4141
          log orders   -3.187       -3.187    15        0.0228        0.0207  0.659   0.744   0.1612
       log orders, d1   -3.858       -3.858    16        0.0029        0.0024 -0.391   0.924   0.1309
      log orders, d12   -2.818       -2.818    15        0.0585        0.0558  0.937   0.018   0.0917
  log orders, d1 d12   -6.428       -6.428    13      < 0.0001        0.0000 -0.143  -0.439   0.0327
largest difference from statsmodels' adfuller: 1.0e-11; lags chosen: the same in every row
d1: each month minus the one before; d12: minus the same month a year before; std: the transformed series' spread
Actual output from this lesson's run.
```

What it means

Every statistic and every chosen lag matches statsmodels (largest difference 1.0e-11), and the simulated p-values are close to MacKinnon's (0.0228 against 0.0207 for the log orders).

CO2's level has a unit root by any reading (ADF 2.233). One difference removes it (-4.752), but look at that row's lag-12 autocorrelation: 0.922. The season is untouched, and the ADF test does not see it, because it asks only about a unit root. The seasonal difference d12 removes the season (lag-12 autocorrelation 0.044) and both differences together leave the smallest spread (0.4141).

The log orders are harder: their level already rejects a unit root (-3.187), likely because the 2008–09 crash and recovery look like a series pulled back to its mean (not tested here), while the seasonally differenced orders narrowly fail (-2.818, p 0.0558).

WHY CHOOSE THIS?

A standard, well-understood test with published tables, here reproduced exactly.

WHY NOT THIS? (TRADEOFFS)

It answers one narrow question, a unit root or not; it ignores seasonality and can be fooled by a single large shock.

5. Over-differencing

DEFINITION — start here, no prior knowledge assumed

Differencing a series that is already stationary does harm: the result is more spread out and gains a negative autocorrelation that was not there. For white noise the theory is exact: the spread grows by √2 and the lag-1 autocorrelation becomes –0.5.

REAL-WORLD SCENARIO

Correcting an essay that has no mistakes introduces new ones: each 'fix' is now an error.

ACTUAL OUTPUT

```
white noise, differenced: std 0.998 -> 1.419 (theory: x sqrt(2) = 1.412); acf lag 1 -0.532 (theory -0.5)
     CO2, d1 d12: std 0.4141, acf lag 1 -0.297, lag 12 -0.409, ADF -8.40
  CO2, d1 d1 d12: std 0.6676, acf lag 1 -0.613, lag 12 -0.401, ADF -9.25
 CO2, d1 d12 d12: std 0.6919, acf lag 1 -0.287, lag 12 -0.603, ADF -9.56
each extra difference raises the spread and makes one lag's autocorrelation more negative; the ADF test still 'passes',
so it cannot warn about this; the spread and the acf can
Actual output from this lesson's run.
```

What it means

White noise behaved as theory says: spread 0.998 -> 1.419 (1.412 expected), lag-1 autocorrelation -0.532. CO2's d1 d12 series has spread 0.4141; one more first difference raises it to 0.6676 and drives the lag-1 autocorrelation to -0.613, one more seasonal difference to 0.6919 and -0.603 at lag 12. Yet the ADF test 'passes' every one of them (-9.25, -9.56): it cannot warn about over-differencing. The spread and the autocorrelation can.

WHY CHOOSE THIS?

The checks are free: compare the spread and the lag-1 and lag-12 autocorrelations before and after each difference.

WHY NOT THIS? (TRADEOFFS)

Where to stop is a judgement; a negative autocorrelation near –0.5 is a warning sign, not a proof.

6. Two tests, opposite questions: KPSS beside ADF

DEFINITION — start here, no prior knowledge assumed

The **KPSS** test starts from the opposite assumption: the series is stationary, and it looks for evidence against that. It adds up the deviations from the mean; if they drift far from 0 (a running total that keeps growing), the series is not stationary. Agreement between the two tests is reassuring; disagreement is information.

REAL-WORLD SCENARIO

Two doctors: one assumes a patient is ill until tests show otherwise, the other assumes healthy until shown ill. When both conclude 'healthy', you can be fairly confident; when they disagree, more tests are needed.

MATHEMATICS

$$KPSS = \Sigma_t S_t^2 / (n^2 \sigma^2), \quad S_t = e_1 + \ldots + e_t, \quad e = y - m$$

$$\sigma^2 = \text{long-run variance: } \Sigma e^2 + 2 \Sigma_{k=1..L} (1 - k/(L+1)) \Sigma e_t e_{t-k}, \text{ divided by } n$$

IMPLEMENTATION

```
def kpss(y, regression="c", nlags=None):
    y = np.asarray(y, float); n = len(y)
    if nlags is None:
        nlags = min(int(np.ceil(12 * (n / 100) ** 0.25)), n - 1)
    if regression == "c":
        e = y - y.mean()
    else:
        X = np.column_stack([np.ones(n), np.arange(1, n + 1)]); e = y - X @ np.linalg.lstsq(X, y,
            rcond=None)[0]
    s2 = (np.sum(e * e) + 2 * sum((1 - k / (nlags + 1)) * np.sum(e[k:] * e[:-k]) for k in range(1,
        nlags + 1))) / n
    return float(np.sum(np.cumsum(e) ** 2) / n ** 2 / s2)
```

ACTUAL OUTPUT

```
ADF's null hypothesis: a unit root (reject when the statistic is below -2.87, 5%, MacKinnon);
KPSS's null hypothesis: stationary around a constant (reject when the statistic is above 0.463, 5%, KPSS's table)
              series      ADF    KPSS  statsmodels   reading
                 CO2    2.233   2.716        2.716   both say not stationary
              CO2, d1   -4.752   0.071        0.071   both say stationary
            CO2, d12   -4.289   1.060        1.060   conflict: both reject
         CO2, d1 d12   -8.398   0.017        0.017   both say stationary
          log orders   -3.187   0.558        0.558   conflict: both reject
     log orders, d12   -2.818   0.245        0.245   conflict: neither rejects
  log orders, d1 d12   -6.428   0.027        0.027   both say stationary
largest difference from statsmodels' kpss: 1.1e-16
Actual output from this lesson's run.
```

What it means

The from-scratch KPSS matches statsmodels (1.1e-16). The tests agree on CO2's level (not stationary) and on both fully differenced series (stationary). They conflict three times. CO2 after only the seasonal difference: ADF rejects a unit root, KPSS rejects stationarity around a constant (1.060), because the yearly growth rate itself drifts (Day 147: 0.86 to 1.58 ppm a year). The log orders' level: both reject. The seasonally differenced orders: neither rejects. Each conflict says the series is somewhere in between, and that is where judgement, plots and forecasting accuracy (Days 149–153) take over.

WHY CHOOSE THIS?

Two tests with opposite starting points catch each other's blind spots.

WHY NOT THIS? (TRADEOFFS)

Two tests at 5% each make conflicting answers common in real data; neither replaces looking at the series.

7. How often the ADF test is right

DEFINITION — start here, no prior knowledge assumed

A test's **power** is how often it correctly rejects when the null hypothesis is false. For an AR(1) with phi just below 1, the series is stationary but behaves almost like a random walk, and the test struggles. The simulation counts how often it calls such series stationary, for three lengths.

REAL-WORLD SCENARIO

A smoke alarm that only goes off for large fires is reliable when it rings, but it misses small ones. The ADF test is like that near a unit root.

ACTUAL OUTPUT

```
2,000 simulated AR(1) series per cell, y_t = phi y_(t-1) + noise, started from their long-run spread;
the share the Dickey-Fuller test (constant, no extra lags) calls stationary at 5%, using a simulated table for each length
 length    phi 0.8    phi 0.9   phi 0.95   phi 0.99    phi 1.0
    100      0.872      0.306      0.120      0.054      0.052
    250      1.000      0.976      0.468      0.068      0.052
    500      1.000      1.000      0.972      0.129      0.052
phi 1.0 is a random walk: there the share is the false-alarm rate, which should be 0.05
Actual output from this lesson's run.
```

What it means

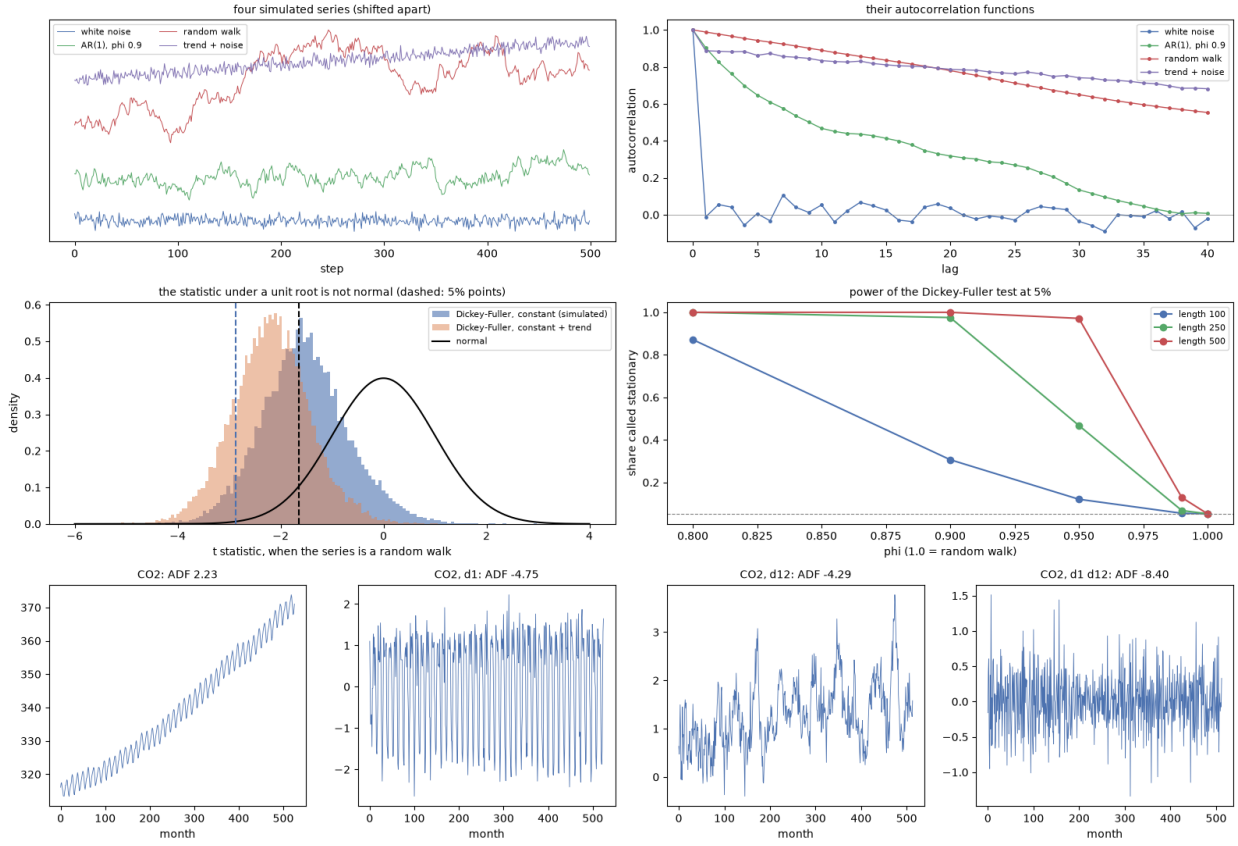
With 100 steps the test recognised phi 0.9 only 0.306 of the time and phi 0.95 0.120; with 500 steps it reached 0.972 for phi 0.95 but still only 0.129 for phi 0.99. On random walks it was wrong 0.052 of the time, as a 5% test should be. So 'no evidence against a unit root' often means 'not enough data', which is one reason practitioners difference by default when in doubt.

WHY CHOOSE THIS?

Knowing the power makes a failure to reject meaningful: with short series, it says little.

WHY NOT THIS? (TRADEOFFS)

The power depends on the true phi, which is unknown; the table is a guide, not a correction.



Top: the four simulated series and their autocorrelation functions. Middle: the Dickey-Fuller statistic for random walks against the normal curve, with both 5% points; the test's power by phi and length. Bottom: CO2, its first, seasonal, and both differences, with each ADF statistic.

Interview preparation — technical questions

Q: How do you test whether a series is stationary?

A: Plot it and its autocorrelation, then run ADF (null: unit root) and KPSS (null: stationary) and read them together; here they conflicted on three of seven series.

Q: Why can't you use the normal table for the Dickey-Fuller statistic?

A: Under a unit root the statistic is not normal: the normal 5% point would wrongly reject 0.452 of random walks, against the simulated -2.874.

Q: What does the 'augmented' in ADF add?

A: Lagged changes in the regression, chosen here by AIC, so short-term memory is not mistaken for mean reversion.

Q: How do you spot over-differencing?

A: The spread rises and the lag-1 autocorrelation goes towards -0.5 or below; CO2's d1 d1 d12 had 0.6676 against 0.4141 and -0.613.

Q: Why might a first difference pass the ADF test but still be unfit for modelling?

A: The test ignores seasonality: CO2's d1 passed (-4.752) with a lag-12 autocorrelation of 0.922.

Interview preparation — theoretical questions

Q: What is a unit root, intuitively?

A: A series in which every shock is permanent, like a random walk; differencing once turns it into the shocks themselves.

Q: Why is the ADF test weak near a unit root?

A: Measured: with 100 steps it detected phi 0.95 only 0.120 of the time. A series with phi 0.95 changes too little in 100 steps to look different from a random walk.

Q: Why do ADF and KPSS have opposite null hypotheses?

A: Each test only controls the error of wrongly rejecting its own null; using both shows when the data cannot decide.

Q: When should you take logs before differencing?

A: When the seasonal swing or the noise grows with the level (Day 147 found this for the orders); differences of logs are then roughly percentage changes.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Stationarity is a property you test for, not one you can see. Differencing removes trend (d1) and season (d12), the ADF and KPSS tests ask the question from opposite sides, and both can be written in a few lines and checked against statsmodels. The tests have limits: ADF needs its own table, ignores seasons, cannot warn about over-differencing and is weak near a unit root. For CO2 the answer is clear: difference once at lag 12 and once at lag 1. That series still has memory, and Day 149 models it.

- ADF and KPSS from scratch match statsmodels (1.0e-11, 1.1e-16).
- Simulated 5% point -2.874, MacKinnon -2.867; the normal table would reject 0.452 of random walks.
- CO2 d1 passes ADF (-4.752) with its season intact (lag 12: 0.922); d1 d12 removes both (spread 0.4141).
- Over-differencing raised CO2's spread to 0.6676 and the ADF test did not notice.
- Power with 100 steps: phi 0.95 detected only 0.120 of the time.

Suggested watch order, if starting from zero

- 1. What is Stationarity (Aric LaBarr) — what stationarity is and why forecasting models need it.
- 2. Time Series Talk: Augmented Dickey Fuller Test + Code (ritvikmath) — the ADF test explained and coded, to compare with today's from-scratch version.

Day 149 preview

- **ARIMA from scratch.** The differenced CO2 series still has memory: its autocorrelation is not 0. Autoregressive and moving-average terms model that memory, the coefficients are fitted by least squares, and forecasts are built back up through the differences, then compared with Day 147's simple forecasts.